

Attack Success Rate (ASR), Worked Out by Hand

ASR is one number with a strict definition — and a confidence interval. We compute per-class and aggregate ASR, derive the interval, and show how many vectors a credible number needs.

What this document does. Red-teaming turns “we thought about it” into “we tested it,” and its headline metric is Attack Success Rate. This document defines ASR precisely, computes it per attack class and in aggregate, derives the confidence interval around it, and shows — with the $\sqrt{n}$ law — how many vectors you need before the number means anything. It closes with the CI/CD gating math.

Concepts covered: ASR numerator/denominator · per-class vs aggregate · the binomial confidence interval · the $\sqrt{n}$ precision law · hard gate vs soft gate · regression detection.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: a red-team suite

The suite has $n = 50$ attack vectors across five classes. A vector *succeeds* if it achieves its objective (e.g. exfiltrates data, escapes the sandbox) against the current build:

class	vectors	successes	class ASR
prompt injection	15	2	0.133
data poisoning	10	1	0.100
sandbox escape	8	0	0.000
exfiltration	12	0	0.000
replay	5	0	0.000
total	50	3	0.060

Intuition

ASR is deliberately blunt: *successes divided by attempts*. Its power is that it is a single comparable number — against last quarter, against a target, against peers. But a single number from few trials is noisy, so the discipline is to always report ASR *with* its uncertainty, never bare.

2 Step 1 — The metric

$$\text{ASR} = \frac{\# \text{successful vectors}}{\# \text{total vectors}}.$$

Aggregate: $\text{ASR} = 3/50 = 0.060 = \mathbf{6.0\%}$. Per class, ASR isolates *where* the defense leaks: prompt injection (13.3%) is the weak spot; sandbox, exfiltration, and replay are clean at 0%.

Watch out

The denominator must be fixed in advance. Quietly adding easy vectors that always fail deflates ASR without improving security — the metric is only honest if the suite is defined by the threat model (Module 4.0), not chosen to look good. “What counts as success” must be written down per vector, too.

3 Step 2 — The confidence interval

ASR is a sample proportion $\hat{p}$ from n Bernoulli trials, so its standard error and (normal-approximation) 95% interval are

$$\text{SE} = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \quad \text{CI}_{95\%} = \hat{p} \pm 1.96 \text{ SE}.$$

With $\hat{p} = 0.06$, $n = 50$:

$$\text{SE} = \sqrt{\frac{0.06 \cdot 0.94}{50}} = \sqrt{0.001128} = 0.0336, \quad \text{CI} = 0.06 \pm 1.96(0.0336) = 0.06 \pm 0.066.$$

That is $[-0.006, 0.126]$, clipped to $[0\%, 12.6\%]$. The interval is *wider than the estimate itself* — with only 50 vectors, “6%” could plausibly be anywhere from 0 to 13%.

4 Step 3 — Precision needs $\sqrt{n}$

The interval half-width is $1.96 \text{ SE} \propto 1/\sqrt{n}$. To halve it you need $4\times$ the vectors:

vectors n	SE	95% half-width
50	0.0336	$\pm 6.6\%$
200	0.0168	$\pm 3.3\%$
800	0.0084	$\pm 1.6\%$
3200	0.0042	$\pm 0.8\%$

Intuition

This is why a credible ASR needs a *large* suite. A handful of vectors gives a number with an error bar so wide it can’t distinguish 6% from 0% or 13%. The $\sqrt{n}$ law is unforgiving: precision is expensive, and “our ASR is 6%” is only meaningful next to “ $\pm$ what, over how many vectors.”

5 Step 4 — Gating CI/CD

Two gates, two kinds of math:

- **Hard gate (regression):** any *critical* vector that flips `blocked` $\rightarrow$ `succeeded` fails the build. This is a per-vector boolean, no statistics — a single regression is unacceptable.
- **Soft gate (threshold):** aggregate ASR above a target (say 5%) warns. Because the CI is wide at small n , compare the *interval* to the threshold: here ASR 6% with CI up to 12.6% clearly may exceed 5%, so the soft gate trips.

6 The argument, at a glance

#	Idea	Formula	Value
1	Aggregate ASR	successes/total	6.0%
1	Worst class	per-class ASR	injection 13.3%
2	Standard error	$\sqrt{\hat{p}(1-\hat{p})/n}$	0.0336
2	95% CI	$\hat{p} \pm 1.96 \text{ SE}$	[0, 12.6%]
3	Precision law	half-width $\propto 1/\sqrt{n}$	$4 \times n \Rightarrow \frac{1}{2}$ width

7 Equation cheat-sheet

Attack Success Rate	$\hat{p} = \text{successes} / \text{total}$
Standard error	$\text{SE} = \sqrt{\hat{p}(1-\hat{p})/n}$
95% interval	$\hat{p} \pm 1.96 \text{ SE}$
Precision scaling	half-width $\propto 1/\sqrt{n}$
Hard gate	any critical regression $\Rightarrow$ fail
Soft gate	CI vs ASR threshold

A Reference implementation

```
import math
def asr(successes, total):
    return successes / total

def ci95(p, n):
    se = math.sqrt(p * (1 - p) / n)
    return se, 1.96 * se

p = asr(3, 50) # 0.06
se, hw = ci95(p, 50)
print(round(p,3), round(se,4), round(hw,3)) # 0.06 0.0336 0.066
for n in (50, 200, 800, 3200):
    print(n, round(ci95(p, n)[1], 4)) # 0.066 0.033 0.016 0.008
```

— end of document —